

MVVM-Entwurfsmuster

MVVM – Model-View-ViewModel

- Trennen von Modell, Präsentationslogik und Oberfläche
 - Entwickler und Designer können unabhängiger voneinander arbeiten
 - Entwickler können Unit Test für ViewModel und Model kreieren, ohne die View zu nutzen
 - Einfacher Austausch des User Interface



Das MVVM (Model-View-ViewModel)-Pattern ist ein Designpattern, das in WPF verwendet wird, um die Trennung von UI-Logik, Anwendungslogik und Daten zu ermöglichen. Es fördert eine klare Strukturierung des Codes und eine bessere Testbarkeit. Das MVVM-Pattern besteht aus den folgenden Komponenten:

1. View: Das View ist die Benutzeroberfläche der Anwendung. Es ist verantwortlich für die Darstellung der Daten und die Interaktion mit dem Benutzer. In WPF besteht die View aus XAML-Dateien, die das visuelle Layout definieren.
2. ViewModel: Das ViewModel ist die Verbindung zwischen der View und dem Model. Es enthält Eigenschaften und Befehle, die von der View gebunden werden, um die Daten anzuzeigen und die Benutzerinteraktionen zu verarbeiten. Das ViewModel interagiert mit dem Model, um Daten abzurufen oder zu speichern und die entsprechenden Updates an die View weiterzugeben.
3. Model: Das Model repräsentiert die Daten und die Geschäftslogik der Anwendung. MVVM betrachtet im Endeffekt alles, was kein View oder ViewModel ist bzw. diesen zuarbeitet, wie z.B. Converter, als Model.

Der Zweck besteht zum Einen darin, die Logik aus dem View in das ViewModel zu verschieben, wodurch Ersteres weitgehend vom Code isoliert bleibt und sich auf das Rendern und die Benutzerinteraktion konzentrieren kann, während das ViewModel die Verarbeitung der Daten und die Steuerung der Anwendungslogik übernimmt. Zum Anderen erfolgt eine Trennung zwischen Präsentationslogik und dem Rest des Programms, so dass im Model eine potenzielle visuelle Darstellung nicht beachtet werden muss.

MVVM

Model:

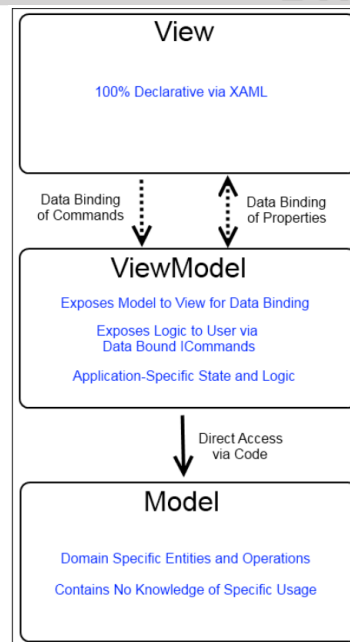
- Set von Klassen welche Daten aus Service oder Datenbank repräsentieren

View:

- Code korrespondierend zur Visuellen Repräsentation der Daten

ViewModel:

- Adapter zwischen View und Model
- Kapselt Daten des Model
- Bereitet Daten für Präsentation und Modifikation durch View auf
- Behandelt Interaktionen mit View



View <-> ViewModel:

Das View kommuniziert mit dem ViewModel über den DataContext und die damit kooperierenden Mechanismen. Pro View existiert ein ViewModel, welches in dessen DataContext liegt. Daten werden über DataBindings ausgetauscht, während Befehle über Commands kommuniziert werden. Das ViewModel hat INotifyPropertyChanged implementiert und verwaltet gegebenenfalls die Validierung. Idealerweise kennt das ViewModel nur lose das verknüpfte View, was unter .NET aber teilweise ohne zusätzliche Mechaniken schwer zu realisieren ist, da das View viele wichtige Mechaniken, wie beispielsweise das Schließen von Fenstern oder das DialogResult verwaltet.

ViewModel <-> Model:

Das ViewModel greift auf das Model über codeseitige Referenzierungen zu. Es benutzt die benötigten Objekte, Eigenschaften und Funktionen der Modelklasse um die benötigten Daten aufzubereiten und an das View weiterzuleiten bzw. Daten aus dem View an das Model zu übergeben. Dazu werden häufig Eigenschaft ohne eigene Felder verwendet, welche direkt in das Model hineingreifen. Wichtig ist, dass das Model keine Referenzierung auf ViewModel oder gar View aufweist, sondern alle Kommunikation vom ViewModel ausgeht.